

### Bonus task 8

#### Modal analysis for a beam structure analyzed with two-dimensional four-node bilinear solid (plane) elements

Possibility to earn six bonus points

Deadline: 28.6.2026

You aim to perform modal analysis within a finite element (FE) framework and to clarify the differences between plane elements using full Gaussian and reduced integration schemes, as well as the effects of different mass matrices in modal analysis. To achieve this, you will solve the eigenfrequencies and mode shapes of free structures using various mass matrices and two-dimensional quadrilateral four-node bilinear solid elements (plane element "Q4") in MATLAB/Python. The mass matrices under consideration are the consistent mass matrix and the lumped mass matrix.

Write a clear report that presents the results of the next tasks:

- Solve the lowest eight eigenfrequencies and mode shapes of the free structure analyzed with a  $1 \times 1$  mesh, using the consistent mass matrix and a stiffness matrix based on reduced integration. Categorize the modes as accurately as possible (e.g., rigid body, bending, shear, transversal, etc.). Present the eigenfrequencies in a table and visualize the mode shapes in the report. **Note: A  $1 \times 1$  mesh is sufficient for this step.** (2 points)
- Derive the lumped mass matrix and verify it against the consistent mass matrix in the modal analysis of a free beam. Solve the lowest eight eigenfrequencies and mode shapes using  $1 \times 1$ ,  $5 \times 1$ ,  $20 \times 4$ , and  $40 \times 8$  meshes. Present the percentage difference (%) between the mass matrices for the eigenfrequencies, ensuring that you compare the same eigenmodes and frequencies. Discuss what you observe about the mass matrices as the mesh becomes finer and finer in the modal analysis. (3 points)

Submit the report in pdf format. Submit also all needed MATLAB/Python files (m-files.py) and check they run without problems.

### Tips

To solve the problem, run the main code `run_ModalAnalysis_Q4_v21.m` in Matlab or `run_ModalAnalysis_Q4_v21.py` in Python. To derive the lumped mass matrix, you could think that mass is "lumped" at the nodes in equal parts. The lumped mass matrix should fulfill the requirement of mass preservation as follows:

$$\sum_i M_{ii} = \rho \int_V dV = \rho W \int_{-1}^1 \int_{-1}^1 \det \mathbf{J}_e d\xi d\eta \quad (1)$$

where  $M_{ii}$  are (diagonal) components of the lumped mass matrix. Note that lumped mass matrix has only diagonal values and therefore, it gives some computational savings in linear algebra.

Therefore, the lumped mass matrix can be found:

$$\mathbf{M}_{lumped} = \alpha \mathbf{I} \rho W \int_{-1}^1 \int_{-1}^1 \det \mathbf{J}_e d\xi d\eta \quad (2)$$

where  $\alpha$  is a factor that defines how much mass is lumped at the nodes and  $\mathbf{I}$  is a unit matrix of size  $8 \times 8$

In the MATLAB/Python code, you could derive this by modifying alpha in `run_DeriveQ4psR_v12_MassMatrices_template.m` / `run_DeriveQ4psR_v12_MassMatrices_template.py` file. Run the code before running the main code.

```
% Lumped mass matrix
Mlumpedtmp=zeros(8,8);
alpha=1;
Mlumpedtmp=alpha*rho*Lz*diag(ones(8,1));      % you need to think how much mass should ...
        be lumped at nodes...so this line must be modified by multiplying it by the ...
        factor alpha. Should be different than 1.
detJeint=int(int(detJe,xi,-1,1),eta,-1,1);

Mlolumbed=Mlumpedtmp*detJeint;
matlabFunction(Mlolumbed,'file','MlolumpedQ4','vars',{X,rho,Lz});
```